Extension de la bibliothèque automata-lib

Récapitulatif complet : tout ce que nous avons fait, depuis le début, expliqué pas à pas

Mouhamed Diop — L2 Informatique
Université Grenoble Alpes

État mesuré : 62/62 algorithmes · 149 tests verts · 97 % de couverture · ruff 0 erreur · mypy OK

Sommaire

1. Le point de départ : le sujet et les automates 3

2. Mise en place de l'environnement 5

3. Le plan et l'architecture 6

4. Les 62 algorithmes, expliqués 9

5. Les tests 13

6. Les bugs trouvés et ce qu'ils nous ont appris 15

7. Qualité du code : PEP8, typage, documentation, git 16

8. Chronologie complète du projet 18

9. Aide-mémoire : les commandes utiles 19

10. Limites connues et suite possible 20

11. Glossaire 21

1. Le point de départ : le sujet et les automates

1.1 Ce que le professeur demande

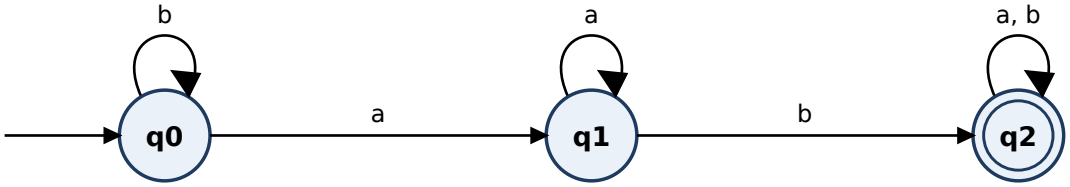
Le sujet (un PDF de 29 pages) s'intitule « **Extension de la bibliothèque automata-lib — Langages, Automates, Grammaires et Extensions Python** ». C'est un projet pédagogique de L2 Informatique, à rendre au bout de trois mois. Il demande de construire un **paquet Python** qui **ajoute** des algorithmes à une bibliothèque existante, **automata-lib**, sans jamais modifier cette dernière.

Exigence du sujet	Ce que ça veut dire
Ne jamais modifier automata-lib (§4.2)	On crée notre propre paquet qui *étend* la bibliothèque (version figée 9.2.0), au lieu de la modifier.
Héritage + mixins (§3.2, §4.3)	Les algorithmes sont rangés dans des classes thématiques (mixins) combinées par héritage multiple.
Immuabilité (§4.2)	Chaque méthode renvoie un **nouvel** automate ; l'automate d'origine n'est jamais changé.
62 algorithmes en 3 niveaux (§4.1)	Niveau 1 : bases. Niveau 2 : opérations sur les langages. Niveau 3 : grammaires et théorie algébrique.
Tests pytest (§4.4)	Des tests automatiques pour chaque algorithme.
Packaging (§4.5)	`pip install -e .` doit fonctionner (fichier pyproject.toml).
Documentation (§4.6)	README, exemples, docstrings, complexité de chaque algorithme.
PEP8 + typing + FrozenSet (§4.2)	Style de code standard et annotations de types.
Compte rendu (§6)	Une synthèse finale détaillée du travail réalisé.

1.2 Qu'est-ce qu'un automate fini ? (rappel simple)

Un **automate fini** est une petite machine qui lit un mot lettre par lettre et répond « accepté » ou « refusé ». Il est défini par cinq éléments :

- un ensemble d'**états** Q (les « positions » de la machine) ;
- un **alphabet** Σ (les lettres autorisées, par exemple {a, b}) ;
- des **transitions** (« si je suis dans l'état p et que je lis la lettre a, je vais dans l'état q ») ;
- un **état initial** (où l'on commence) ;
- des **états finaux** (si on s'arrête dedans, le mot est accepté).



Langage reconnu : les mots qui contiennent « ab ». q0 = état initial (flèche entrante), q2 = état final (double cercle).

Figure 1 — Un automate fini déterministe (DFA) à trois états.

L'ensemble des mots acceptés s'appelle le **langage** de l'automate. Ici, le mot « bab » est accepté (on passe par q0, q0, q1, q2) alors que « bba » est refusé.

DFA, NFA et transitions epsilon

Notion	Explication	Exemple
DFA (déterministe)	Dans chaque état, pour chaque lettre, **une seule** transition possible.	q0 --a--> q1 (jamais deux choix)
NFA (non déterministe)	Plusieurs transitions possibles pour la même lettre : la machine « essaie tous les chemins ».	q0 --a--> {q1, q2}
Transition epsilon (ε)	Transition qui ne consomme **aucune** lettre. Dans notre code, epsilon est représenté par la chaîne vide `''`.	q0 --ε--> q1
Automate partiel	Certaines transitions manquent. Convention : une transition manquante mène à un **puits implicite** (état poubelle non final).	pas de transition b depuis q1

1.3 Qu'est-ce que automata-lib ?

automata-lib est une bibliothèque Python qui fournit les classes `DFA`, `NFA`, `GNFA` (automates généralisés) et quelques opérations. Notre travail consiste à y **ajouter** les algorithmes manquants. Point important découvert en cours de route : la version 9.2.0 est **plus riche** que ce que décrit le sujet (elle contient déjà `complement`, `union`, `minify`...). Cela a eu des conséquences sur nos choix (voir §4.4).

2. Mise en place de l'environnement

2.1 Installation sur Mac

Tout se passe dans le dossier `~/Documents/automate` avec un **environnement virtuel** (venv) appelé `automate-env`. Un venv est une copie isolée de Python où l'on installe les bibliothèques du projet sans toucher au reste du Mac.

```
cd ~/Documents/automate
source automate-env/bin/activate      # active l'environnement (le préfixe
                                      # (automate-env) apparaît dans le terminal)
pip install automata-lib==9.2.0       # version figée par le sujet
```

2.2 Les obstacles rencontrés

Problème	Cause	Solution
***metadata-generation-failed* / échec de pycairo**	pycairo a besoin de bibliothèques système (cairo, pkg-config) pour être compilé.	`brew install pkg-config cairo` (+ graphviz), puis réinstallation.
matplotlib / automata « introuvables »	Commandes lancées hors du venv, ou paquets non installés.	Activer le venv puis `pip install`.
Scripts de setup du professeur (Windows, conda)	Pensés pour un autre système ; non pertinents sur Mac.	Ignorés ; installation manuelle avec Homebrew + pip.
Terminal quitté	Le venv se désactive à la fermeture.	Refaire `source automate-env/bin/activate` à chaque nouvelle session.

Résultat : environnement fonctionnel (graphviz, pygraphviz, matplotlib, ipython), et un premier automate de test affiché avec succès.

2.3 Les outils installés au fil du projet

Outil	Rôle
pytest	Lance les tests automatiques.
pytest-cov	Mesure la **couverture** : quelle part du code est exécutée par les tests.
hypothesis	Génère des automates aléatoires pour tester des propriétés mathématiques (property-based testing).
ruff	Vérifie le style PEP8 et reformate le code.
mypy	Vérifie les annotations de types.
git	Historique des modifications : un commit par étape.

3. Le plan et l'architecture

3.1 Organisation du dépôt

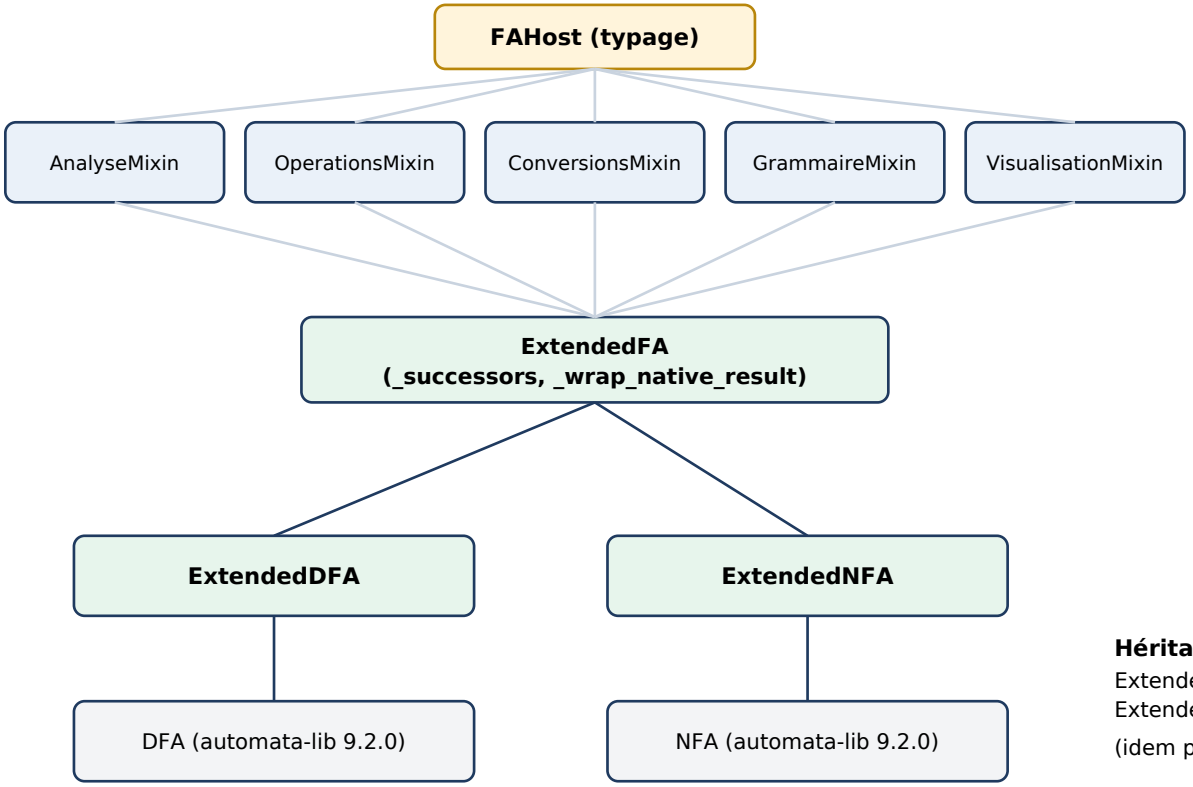
```
automata_extensions/
├── regex_ast.py              AST de regex, dérivées, lemme d'Arden
├── fa/
│   ├── base.py               ExtendedFA : colle les 5 mixins ensemble
│   ├── dfa.py                ExtendedDFA(ExtendedFA, DFA)
│   ├── nfa.py                ExtendedNFA(ExtendedFA, NFA)
│   └── mixins/
│       ├── _host.py          FAHost : déclarations pour le typage
│       ├── analyse.py        parcours, accessibilité, cycles, SCC
│       ├── operations.py     langages, produit, minimisation, opérations
│       ├── conversions.py    epsilon, déterminisation, regex <-> automate
│       ├── grammaire.py      grammaires, dérivations, résiduels, monoïde
│       └── visualisation.py  export DOT / SVG / PDF
└── grammar/
    ├── grammar.py            Grammar, RegularGrammar
    └── parse_tree.py         Leaf, Node (chaîne de dérivation)
tests/   conftest.py  strategies.py  test_*.py (8 fichiers)
pyproject.toml   README.md
```

3.2 Mixins et héritage multiple : l'idée

Plutôt que d'écrire 62 méthodes dans une seule classe géante, on les range par thème dans des **mixins**. Un mixin est une classe qui apporte des méthodes mais qui n'est pas destinée à être utilisée seule. On les assemble ensuite par **héritage multiple** :

```
class ExtendedFA(AnalyseMixin, OperationsMixin, ConversionsMixin,
                 GrammaireMixin, VisualisationMixin): ...

class ExtendedDFA(ExtendedFA, DFA): ...   # nos algos + les méthodes natives
class ExtendedNFA(ExtendedFA, NFA): ...
```



Héritage multiple : ExtendedDFA hérite de ExtendedFA ET de DFA (idem pour le NFA).

Figure 2 — Architecture : mixins → ExtendedFA → ExtendedDFA / ExtendedNFA.

Conséquence agréable : un `ExtendedDFA` possède à la fois les méthodes de automata-lib *et* les nôtres, et les appels se **chaînent** : `dfa.union(autre).trim().minimize()`.

3.3 Les deux mécanismes centraux

_successors(state, symbol) : unifier DFA et NFA

Dans un DFA, `transitions[état][lettre]` est un **seul état** ; dans un NFA c'est un **ensemble** d'états. Pour que chaque algorithme n'ait jamais à se demander à quel type d'automate il a affaire, `_successors` renvoie toujours un **ensemble** d'états (vide s'il n'y a pas de transition).

_wrap_native_result(automaton) : rester « chaînable »

Quand on délègue à automata-lib (par exemple pour `complement`), le résultat est un `DFA` natif, qui n'a pas nos méthodes. `_wrap_native_result` le reconstruit en `ExtendedDFA` ou `ExtendedNFA` pour que nos méthodes restent disponibles. Un bug important est lié à cette fonction (voir §6).

L'epsilon et la méthode « une cible, plusieurs cibles »

Le symbole epsilon (`''`) n'est pas dans l'alphabet. Les parcours doivent donc itérer sur `self.transitions.get(état, {})` (qui contient les epsilon) et non sur l'alphabet — sinon les transitions epsilon sont ignorées (erreur rencontrée et corrigée).

3.4 Principes de conception

- **Immuabilité** : toute méthode renvoie un nouvel objet.
- **Wrapper plutôt que réimplémenter** si automata-lib sait déjà le faire (§4.2 du sujet).
- **Renommer en cas de conflit de sens** avec la bibliothèque.
- **Lisibilité d'abord** : bibliothèque pédagogique, pas d'optimisation prématurée.

- **Docstrings** avec les rubriques Returns, Complexity, References.

3.4 bis Vérifier avant d'implémenter : la méthode hasattr

Avant d'écrire une méthode, on vérifiait avec `hasattr(DFA, 'nom')` si automata-lib l'a déjà. Résultats de ces vérifications (sur automata-lib 9.2.0) :

Méthode	DFA	NFA	Décision
complement, predecessor(s), minify	oui	—	complement : wrapper. predecessors : sens différent, on a renommé la nôtre *predecessor_states*.
union, intersection	oui	oui	Wrappers.
difference, issubset…	oui	non	difference : wrapper (déterminisation préalable pour un NFA).
concatenate, kleene_star, reverse	non	oui	Wrappers (un DFA est d'abord vu comme un NFA).
left_quotient, right_quotient	non	oui (prend un **automate**)	Collision de sens : les nôtres s'appellent *left_quotient_word* / *right_quotient_word* (ils prennent un mot).
cartesian_product, language_equals, language_subset, is_isomorphic	non	non	Implémentation libre.

> **Un document trompeur écarté**
> Un PDF fourni (« automata_documentation_v1.pdf ») prétendait décrire une implémentation complète alternative. Il contredisait le sujet officiel et présentait des signes de contenu inventé. Décision : ne retenir que ce qui est **vérifiable par introspection** dans ton environnement (hasattr, inspect.signature), et écarter le reste.

4. Les 62 algorithmes, expliqués

Notation : |Q| = nombre d'états, |E| = nombre de transitions, |Σ| = taille de l'alphabet. Toutes les méthodes renvoient de nouveaux objets (jamais de modification de l'automate d'origine). Les complexités viennent des docstrings du code.

4.1 Niveau 1 — Parcours, accessibilité, structure

Méthode	Ce qu'elle fait	Complexité
dfs(start)	Parcours en profondeur **itératif** avec une pile (évite la récursion). Renvoie les états dans l'ordre de première visite.	O(|Q|+|E|)
bfs(start)	Parcours en largeur avec une file : les états les plus proches de l'initial d'abord.	O(|Q|+|E|)
predecessor_states(état)	Les états qui ont au moins une transition vers l'état donné (renommée pour ne pas entrer en collision avec *predecessors* natif, qui travaille sur des mots).	O(|E|)
accessible_states / is_accessible	États atteignables depuis l'initial (c'est un dfs).	O(|Q|+|E|)
reachable_states_from(état)	Idem, mais depuis un état quelconque.	O(|Q|+|E|)
coaccessible_states / is_coaccessible	États depuis lesquels on peut atteindre un état final. Astuce : ce sont les états *accessibles* de l'automate **renversé**.	O(|Q|+|E|)
useful_states	Accessibles **et** coaccessibles (intersection).	O(|Q|+|E|)
induced_subautomaton(sous-ensemble, initial)	Restreint l'automate à un sous-ensemble d'états (les transitions sortantes sont retirées). Peut changer l'état initial.	O(|Q|+|E|)
trim / is_trim	Émonde : ne garde que les états utiles. **Cas limite** : si le langage est vide, renvoie un automate à 1 état non final (un automate a toujours un état initial).	O(|Q|+|E|)
strongly_connected_components	Composantes fortement connexes par l'algorithme de **Tarjan** (1972) : groupes d'états mutuellement atteignables.	O(|Q|+|E|)
has_cycle / has_cycle_from	Cycle = une composante de taille > 1, ou un état avec une boucle sur lui-même.	O(|Q|+|E|)
is_language_empty	Vide ssi aucun état final n'est accessible.	O(|Q|+|E|)
is_language_infinite	Infini ssi il existe un cycle à la fois accessible et coaccessible (on peut « pomper » indéfiniment).	O(|Q|+|E|)
accepts_word	Wrapper de *accepts_input* natif.	O(|w|·|Q|)
execution_trace(mot)	La suite des états (ou ensembles d'états pour un NFA, avec fermeture epsilon) visités en lisant le mot.	O(|w|·|Q|)

4.2 Niveau 1 — Complétion, langages, minimisation

Méthode	Ce qu'elle fait	Complexité
is_complete / complete	Complet = chaque état a une transition pour chaque lettre. *complete* ajoute un état **puits** `__sink__` (le nom s'allonge si déjà pris).	O(|Q|·|Σ|)

Méthode	Ce qu'elle fait	Complexité
complement	Wrapper du natif : le complémentaire reconnaît exactement les mots refusés.	natif
cartesian_product(autre, prédicat)	Produit synchronisé de deux automates, **paresseux** (on ne visite que les couples atteignables). Le prédicat final décide l'opération (par défaut : intersection).	O(|Q1|·|Q2|)
language_subset	L(A) ⊆ L(B) ssi L(A) ∩ complément(L(B)) est vide.	O(|Q1|·|Q2|)
language_equals	Égalité de langages = double inclusion.	O(|Q1|·|Q2|)
is_isomorphic	Mêmes automates à un renommage d'états près (plus fort que l'égalité de langage). Parcours synchronisé qui construit la correspondance état-à-état.	O(|Q|+|E|)
is_prefix_closed	Langage clos par préfixe ssi tous les états **utiles** sont finaux.	O(|Q|+|E|)
prefix_closed_sublanguage	Plus grand sous-langage clos par préfixe : on ne traverse jamais un état non final.	O(|Q|+|E|)
distinguishable_states	Remplissage de table : deux états sont distinguables si un mot mène l'un à un final et l'autre non. Une transition manquante est un saut vers un **puits implicite** (sentinelle `SINK`).	O(|Q|²·|Σ|)
myhill_nerode_partition	Classes d'états équivalents (non distinguables deux à deux).	O(|Q|²·|Σ|)
is_minimal / minimize	Minimal = accessible et classes singletons. *minimize* : trim puis fusion des états équivalents (version pédagogique voulue par le sujet).	O(|Q|²·|Σ|)
is_deterministic	Au plus une cible par lettre et aucun epsilon.	O(|Q|·|Σ|)

4.3 Niveau 1 — Epsilon, déterminisation, export

Méthode	Ce qu'elle fait	Complexité
epsilon_closure	Ensemble des états atteignables en ne suivant que des transitions epsilon.	O(|Q|+|E|)
reverse	Automate renversé : transitions inversées, initial et finaux échangés. Un nouvel état initial est relié par epsilon aux anciens finaux (résultat : un NFA). Sur un NFA, wrapper du natif.	O(|Q|+|E|)
determinize	NFA → DFA par **construction des sous-ensembles**. Paresseuse : on ne crée que les sous-ensembles atteints.	$O(2^{\|Q\|})$ pire cas
remove_epsilon_transitions	NFA équivalent sans epsilon (wrapper de *eliminate_lambda*) ; copie pour un DFA.	O(|Q|²·|Σ|)
is_universal_language	L = Σ* ssi le complément est vide.	O(|Q|·|Σ|)
eliminate_epsilon_states_normalized	Élimination directe des epsilon : on ne traite p --ε--> q que lorsque q n'a plus d'epsilon sortant.	O(|Q|³)
export_dot / export_svg / export_pdf	Export Graphviz du schéma de l'automate. *export_tikz* lève volontairement `NotImplementedError` (nécessite dot2tex).	O(|Q|+|E|)

4.4 Niveau 2 — Opérations sur les langages et conversions

Méthode	Ce qu'elle fait	Choix
union, intersection	Wrappers des méthodes natives (DFA et NFA).	wrapper
difference	Natif seulement sur DFA : si un opérande est un NFA, on le détermine d'abord.	wrapper
concatenate, kleene_star	Natifs sur NFA seulement : un DFA est d'abord converti en NFA (*NFA.from_dfa*).	wrapper
left_quotient_word(mot)	Quotient à gauche : {u : mot·u ∈ L}. On lit le mot depuis l'initial ; les états atteints deviennent les nouveaux initiaux (avec fermeture epsilon pour un NFA).	propre
right_quotient_word(mot)	Quotient à droite : {u : u·mot ∈ L}. Réutilise le code existant : reverse → left_quotient_word(mot renversé) → reverse.	réutilise
regex_to_nfa	Construction de **Thompson** (wrapper de *NFA.from_regex*).	wrapper
to_regex_state_elimination	Automate → expression régulière par élimination d'états (wrapper de GNFA).	wrapper
brzozowski_derivative(regex, a)	Dérivée de Brzozowski : $D_a(L) = \{w : a \cdot w \in L\}$, calculée directement sur l'arbre de la regex.	propre
brzozowski_automaton	Automate dont les états sont les dérivées **distinctes** de la regex. Termine grâce aux constructeurs « intelligents » qui simplifient (r|r = r, ε·r = r…).	propre
to_regex_arden	Automate → regex en résolvant le **système d'équations** de langages (un par état) avec le lemme d'Arden.	propre

Le fichier regex_ast.py
Contient l'arbre syntaxique des regex : classes **Empty** (∅), **Epsilon** (ε), **Symbol**, **Union** (représentée comme un *ensemble* : r|r et r deviennent identiques), **Concat**, **Star**. Les fonctions *mk_union / mk_concat / mk_star* simplifient à la construction (∅ absorbant en concaténation, ε neutre, (r*)* = r*). C'est ce qui permet à l'automate de Brzozowski d'avoir un nombre d'états fini et petit. S'y trouvent aussi *parse_regex*, *is_nullable*, *derivative*, *arden_lemma* et *solve_equation_system*.

Le lemme d'Arden en deux phrases

Si X = A·X + B et que A ne contient pas le mot vide, alors **X = A*·B**. Pour un automate, on écrit une équation par état (« mots acceptés depuis cet état ») puis on élimine les états un à un par substitution, en appliquant Arden à chaque auto-référence. **Piège rencontré** : distinguer la constante *None* (état non acceptant : langage vide) de la constante '' (état acceptant : contient le mot vide).

4.5 Niveau 3 — Grammaires et théorie algébrique

Méthode	Ce qu'elle fait	Complexité
to_grammar	Automate → grammaire régulière (linéaire à droite) : un non-terminal par état, une règle *p* → *a q* par transition, *p* → *ε* pour chaque état final.	O(|Q|+|E|)
grammar_to_automaton	Construction inverse. Un état « accepteur » est ajouté pour les règles *p* → *a* qui terminent le mot.	O(|règles|)
derivation_tree(mot)	Chaîne de dérivation (Node/Leaf) : comme la grammaire est linéaire à droite, l'arbre est une chaîne. *sequence()* affiche 1 ⇒ a2 ⇒ a.	O(|w|)

Méthode	Ce qu'elle fait	Complexité
arden_lemma / solve_equation_system	Résolution d'une équation / d'un système d'équations rationnelles (voir §4.4).	$O(n^3)$
myhill_nerode_language_classes	Classes de quotients du **langage** (un par état du DFA minimal), représentées par le plus court mot qui y mène (BFS). À ne pas confondre avec *myhill_nerode_partition* qui classe les **états**.	$O(\|Q\|^2 \cdot \|\Sigma\|)$
residual_automaton	Automate dont les états sont les résiduels (nommés par leur mot représentatif). Isomorphe au DFA minimal.	$O(\|Q\|^3 \cdot \|\Sigma\|)$
automaton_morphism(autre, mapping)	Teste si une application entre états préserve initial, finaux et transitions.	$O(\|Q\|+\|E\|)$
syntactic_monoid	Monoïde syntaxique : ensemble des transformations d'états induites par les mots, fermé par composition. Pas d'état puits explicite : *None* représente « bloqué ».	exponentiel (théorique)

4.6 Extension : algorithme de Hopcroft

`minimize_hopcroft()` minimise un DFA par **raffinement de partition**. On part de {finaux, non-finaux} ; pour chaque « séparateur » (bloc, lettre), on éclate les blocs dont les états se comportent différemment. L'astuce qui donne $O(|Q| \log|Q| \cdot |\Sigma|)$: après un éclatement, on n'ajoute à la file que la **plus petite** moitié. Dans notre code, la file est une liste Python, donc ses *remove()* sont en O(n) : simplification pédagogique assumée.

Son vrai intérêt : c'est un **deuxième algorithme indépendant** pour le même problème que *minimize()*. En les comparant sur des automates aléatoires (differential testing), on a détecté un bug dans *minimize()* (voir §6).

5. Les tests

5.1 pytest : le principe

Chaque algorithme a des tests dans le dossier `tests/`. Un test construit un petit automate, appelle la méthode, et vérifie le résultat attendu avec `assert`. Les automates fréquemment utilisés sont des **fixtures** dans `conftest.py` (*make_simple_dfa*, *make_epsilon_nfa*). Pour les fichiers, on utilise la fixture *tmp_path* (dossier temporaire).

```
pytest -q                # lance tout : 149 passed
pytest tests/test_analyse.py -q      # un seul fichier
```

Fichier de tests	Contenu
test_analyse.py	Parcours, accessibilité, SCC, cycles, trim... (32)
test_conversions.py	epsilon, déterminisation, regex... (19)
test_grammar.py	Grammaires, dérivations, résiduels, morphismes (21)
test_operations.py	Produit, inclusion, minimisation, quotients... (46)
test_regex.py	AST et dérivées de Brzozowski (10)
test_visualisation.py	Exports DOT/SVG/PDF (4)
test_properties.py	16 propriétés mathématiques (hypothesis)
test_base.py	_wrap_native_result (1)

5.2 hypothesis : tester des propriétés mathématiques

Au lieu de choisir quelques exemples à la main, **hypothesis** génère des centaines d'automates aléatoires et vérifie qu'une propriété reste vraie. Si elle échoue, hypothesis réduit automatiquement le contre-exemple au plus petit possible. Les stratégies (`dfas()`, `words`) sont dans `tests/strategies.py`. Chaque propriété est testée sur **200 exemples**.

Propriété vérifiée	Intérêt
minimize est idempotent et préserve le langage	Minimiser deux fois ne change rien.
complement(complement(A)) = A	Involution du complément.
union et intersection commutatives	A ∪ B = B ∪ A.
accessible_states ⊆ states	Cohérence de base.
trim donne toujours un automate émondé (sauf langage vide)	A révélé le bug du langage vide.
minimize_hopcroft et minimize donnent des résultats isomorphes	Differential testing : a révélé le bug de distinguishable_states.
les résultats de minimisation sont bien minimaux	Vérifie la minimalité.

5.3 La couverture : mesurer ce que les tests exécutent

```
pytest --cov=automata_extensions --cov-report=term-missing -q
```

Fichier	Couverture
grammar.py, visualisation.py, _host.py, dfa.py, nfa.py	100 %

Fichier	Couverture
analyse.py, conversions.py	99 %
grammaire.py	98 %
operations.py, parse_tree.py	97 %
base.py	96 %
regex_ast.py	94 %
TOTAL (1094 instructions, 31 non couvertes)	**97 %**

Les lignes restantes sont surtout des branches défensives et des *__eq__* / *__hash__* déjà utilisés indirectement. La couverture a aussi servi à **trouver du code mort** : une garde jamais fausse dans *is_language_infinite* et une garde jamais vraie dans *determinize*, supprimées.

6. Les bugs trouvés et ce qu'ils nous ont appris

Bug	Comment il a été trouvé	Correction / leçon
_wrap_native_result reconstruisait avec *type(self)* ; or *concatenate* et *kleene_star* natifs renvoient toujours un NFA, même depuis un DFA.	En testant des cas DFA et NFA.	Choisir la classe selon le type **réel** du résultat natif (*isinstance*). Toujours tester DFA *et* NFA.
trim() plantait sur un langage vide (aucun état utile : impossible de construire un automate à 0 état).	**hypothesis** — les 112 tests écrits à la main ne le voyaient pas.	Cas particulier : automate à 1 état *__empty_language__*. Précision théorique : un automate de langage vide n'est jamais « trim » au sens strict ; c'est un cas limite assumé.
distinguishable_states ignorait les transitions manquantes alors qu'elles mènent à un puits implicite non final : des états distinguables étaient fusionnés et *minimize()* donnait un langage faux.	Differential testing : Hopcroft (correct) servait d'oracle contre l'ancien *minimize()*.	Sentinelle **SINK** pendant le calcul, retirée du résultat. Tests de non-régression fixes ajoutés.
Hopcroft : construction finale sur un DFA partiel.	Tests.	Ne remplir que les transitions existantes ; allow_partial=True.
Lemme d'Arden : confusion entre « langage vide » et « mot vide ».	Tests de to_regex_arden.	const=None (vide) ≠ const='' (ε).
Wrappers de operations.py : n'acceptaient pas *retain_names* et *minify* du *DFA.union* natif. *ExtendedDFA(...).union(x, minify=False)* plantait.	**mypy** a signalé une signature incompatible avec la classe parente (violation du principe de Liskov).	Accepter et transmettre ****kwargs**. Vérifié par exécution. Un vrai défaut de comportement, pas seulement de typage.
Tests « faussement verts » : *is_isomorphic* sur alphabets différents passait parce que le nombre d'états était comparé avant l'alphabet.	Lecture critique de la couverture.	Test refait avec deux DFA à 3 états. Toujours vérifier qu'un test échoue pour la bonne raison.
Erreurs de manipulation : imports relatifs avec le mauvais nombre de points, indentation à 8 espaces, fonctions dupliquées, tests jamais ajoutés.	Erreurs à l'exécution, *grep -n*, garde-fou des scripts.	Compter les niveaux de dossiers ; passer aux scripts de remplacement qui comptent les occurrences et refusent d'agir en cas d'ambiguïté.

Le message à retenir pour le compte rendu

Les tests écrits à la main vérifient ce *que tu as pensé* à tester. Les tests par propriétés et la comparaison entre deux algorithmes indépendants trouvent ce *à quoi tu n'as pas pensé* : deux bugs réels ont été découverts ainsi, qu'aucun des 112 tests manuels n'avait détectés.

7. Qualité du code : PEP8, typage, documentation, git

7.1 PEP8 avec ruff

Configuration dans `pyproject.toml` : règles E, W, F (erreurs de style, avertissements, erreurs logiques), lignes de 79 caractères, guillemets conservés. Déroulé :

Étape	Résultat
Mesure initiale	75 problèmes.
`ruff check --fix` **+ remplacement d'une lambda par une vraie fonction (E731)**	Il reste 66 lignes trop longues (E501).
`ruff format`	16 fichiers reformatés ; 16 lignes restantes (commentaires, docstrings).
Raccourcissement à la main (script de remplacement)	**All checks passed!** — 149 tests toujours verts.

7.2 Typage avec mypy (exigence §4.2)

Toutes les fonctions sont annotées ; `mypy` est configuré dans `pyproject.toml` (*disallow_untyped_defs* : une fonction non annotée est une erreur ; *warn_unused_ignores* : un *type: ignore* inutile est signalé). Résultat : **Success: no issues found in 16 source files**.

Point méthodologique : un premier passage de mypy affichait 0 erreur sans rien prouver, car **mypy ne vérifie pas les fonctions non annotées**. L'option *--disallow-untyped-defs* a fait apparaître 123 fonctions à annoter ; on les a traitées fichier par fichier (123 → 71 → 61 → 41 → 30 → 6 → 0).

Le problème des mixins et la solution FAHost

Un mixin utilise `self.states`, `self.transitions`... qui n'existent que dans la classe finale. Vu seul, le mixin ne peut pas être vérifié par mypy. Solution : une classe **FAHost** (*mixins/_host.py*), dont héritent tous les mixins, qui déclare ces attributs (type *Any*) et, **sous TYPE_CHECKING uniquement**, un *__getattr__* pour les méthodes des autres mixins. Ce sont de simples annotations : **aucun effet à l'exécution**.

> **Compromis assumés (à mentionner dans le compte rendu)**
> • Le *__getattr__* rend le contrôle moins strict sur les accès internes entre mixins (une faute de frappe sur un nom de méthode ne serait pas vue par mypy, mais bien par les tests).
> • Quatre `type: ignore[misc]` ciblés sur *super().complement / union / intersection / reverse* : dans un mixin isolé, *super()* pointe vers FAHost ; à l'exécution, l'ordre de résolution (MRO) l'envoie bien vers DFA/NFA — les 149 tests le prouvent.
> • Les wrappers de méthodes natives sont annotés *Any* pour rester compatibles avec la signature de la classe parente ; les autres méthodes qui construisent un automate renvoient "ExtendedFA".

7.3 Documentation et organisation

- **README.md** (101 lignes) : installation, utilisation, architecture, principes de conception, tests, limites connues.
- **Docstrings** sur chaque méthode (Parameters, Returns, Complexity, References).

- **Tests scindés** : le fichier unique de plus de 1000 lignes a été découpé par script en 8 fichiers thématiques ; le garde-fou du script a détecté un test dupliqué (supprimé).
- **pyproject.toml** : build setuptools, dépendance *automata-lib==9.2.0*, extra *dev* (pytest, pytest-cov, hypothesis, ruff, mypy), configuration ruff et mypy.

7.4 git

Chaque étape a été commitée : conformité PEP8, correction des wrappers (**kwargs), typage de *regex_ast.py*, de *grammar.py / parse_tree.py / base.py*, de *visualisation et grammaire*, de *analyse.py*, de *conversions.py*, d'*operations.py* et configuration mypy. Cela permet de revenir à n'importe quelle étape (`git log --oneline`).

8. Chronologie complète du projet

Phase	Ce qui a été fait
1. Découverte	Lecture du sujet (PDF 29 pages), explication de ce qu'est automata-lib, installation sur Mac (brew + pip, pycairo), premier automate affiché.
2. Cadrage	Lecture par pages ; question « comment organiser le projet ? » ; plan des 62 algorithmes en 3 niveaux (document de plan) ; choix de l'architecture mixins ; décision de viser plus que le minimum (extensions difficiles).
3. Fondations	Package *automata_extensions*, ExtendedFA/DFA/NFA, *_successors*, premiers algorithmes d'analyse et premiers tests (~40). Découverte des collisions avec automata-lib (*complement*, *predecessors*).
4. Vérification de la bibliothèque	Contrôles *hasattr* groupés ; renommages ; *_wrap_native_result* ; écartement du PDF de documentation non fiable ; produit cartésien, inclusion, égalité, isomorphisme.
5. Niveau 1 bouclé	Propriétés de langage, minimisation, epsilon, déterminisation, exports (67 puis ~90 tests).
6. Niveau 2 bouclé	Opérations natives wrappées, quotients, Thompson, *regex_ast.py*, Brzozowski, Arden ; bug de *_wrap_native_result* corrigé (90 tests).
7. Niveau 3 bouclé	Grammaires, arbres de dérivation, systèmes d'équations, classes de Myhill-Nerode du langage, résiduels, morphismes, monoïde syntaxique (105 tests) : **62/62**.
8. Property-based testing	hypothesis, *strategies.py*, premières propriétés ; premier bug réel trouvé (*trim* sur langage vide).
9. Hopcroft et differential testing	*minimize_hopcroft* ; deuxième bug réel (*distinguishable_states*) ; 16 propriétés à 200 exemples ; 125 tests.
10. Couverture et finitions	pytest-cov → 97 % ; code mort supprimé ; tests de collisions de noms ; README ; scission des tests (149 tests).
11. Conformité §4.2	ruff (75 problèmes → 0) ; ruff format ; typage de tout le code ; mypy ; FAHost ; correction des wrappers (Liskov) ; configuration mypy.
12. Synthèse	Compte rendu final (§6 du sujet) et ce récapitulatif.

9. Aide-mémoire : les commandes utiles

9.1 Au quotidien

```
cd ~/Documents/automate
source automate-env/bin/activate        # À refaire à chaque nouveau terminal

pytest -q                               # tous les tests (attendu : 149 passed)
pytest tests/test_operations.py -q      # un seul fichier
pytest -k minimize -q                   # tests dont le nom contient « minimize »
pytest --cov=automata_extensions --cov-report=term-missing -q   # couverture

ruff check automata_extensions tests    # style  (All checks passed!)
ruff format automata_extensions tests   # reformate automatiquement
mypy                                    # types  (Success: no issues found)

git status                              # fichiers modifiés
git add -A && git commit -m "message"   # sauvegarder une étape
git log --oneline | head                # derniers commits
```

9.2 Vérifier ce qu'automata-lib propose déjà

```
python -c "
import inspect
from automata.fa.dfa import DFA
print(hasattr(DFA, 'complement'))
print(inspect.signature(DFA.union))
"
```

9.3 Utiliser la bibliothèque

```
from automata_extensions.fa.dfa import ExtendedDFA

dfa = ExtendedDFA(
    states={'q0', 'q1', 'q2'}, input_symbols={'a', 'b'},
    transitions={'q0': {'a': 'q1', 'b': 'q0'},
                 'q1': {'a': 'q1', 'b': 'q2'},
                 'q2': {'a': 'q2', 'b': 'q2'}},
    initial_state='q0', final_states={'q2'})

dfa.accepts_word('bab')                 # True
dfa.trim().minimize()                   # appels chaînés, nouvel automate
dfa.to_regex_arden()                    # expression régulière équivalente
dfa.union(dfa, minify=False)            # options natives acceptées
```

9.4 Pièges de terminal déjà rencontrés

- **macOS** : `sed -i ''` (guillemets obligatoires) ; `cat -e -t` et non `cat -A`.
- **Un collage = une commande** quand elle modifie des fichiers (un *rm* a déjà été collé par erreur avec une autre commande).
- Toujours relancer *ruff*, *mypy* et *pytest* après une modification.
- Après un *nano* : vérifier avec `grep -n` que la modification est bien là, et que l'indentation est de 4 espaces.

10. Limites connues et suite possible

10.1 Limites assumées (à citer dans le compte rendu)

- **export_tikz** non implémenté (nécessite l'outil externe dot2tex) : documenté plutôt que simulé.
- **is_isomorphic** rigoureux pour les DFA uniquement.
- **cartesian_product** ignore les transitions manquantes : correct pour l'intersection (cas d'usage actuel), pas pour un prédicat arbitraire sur automates partiels.
- **residual_automaton** en $O(|Q|^3 \cdot |\Sigma|)$; **syntactic_monoid** potentiellement exponentiel.
- **minimize_hopcroft** : file en liste Python (*remove* en O(n)).
- **Typage** : FAHost, quatre *type: ignore*, wrappers en *Any*.
- 31 lignes non couvertes sur 1094 ; certains tests d'automaton_morphism dépendent de l'ordre d'itération des ensembles.
- Pas de *ExtendedGNFA* (cité dans l'exemple du sujet, hors des 62).

10.2 Suite possible, par ordre de rentabilité

Idée	Intérêt	Effort
Intégration continue GitHub Actions (ruff + mypy + pytest --cov)	Badge vert visible, preuve de qualité automatique.	Faible
Rendre déterministes les tests d'automaton_morphism	Couverture stable, tests plus propres.	Faible
Quelques tests sur les 13 lignes non couvertes de regex_ast.py	Dépasser 97 %.	Faible
Vérifier `pip install -e ".[dev]"` **dans un venv neuf**	Prouve que le packaging fonctionne de zéro.	Faible
Notebook d'exemples (comme Exemples.ipynb du professeur)	Démonstration concrète pour le correcteur.	Moyen
Dérivées partielles d'Antimirov	Algorithme avancé, automate plus petit que Brzozowski.	Moyen
Export TikZ avec dot2tex (dépendance optionnelle)	Supprime une limite documentée.	Moyen
Automates à pile, L* d'Angluin, Büchi, CYK	Extensions très différenciantes mais lourdes.	Élevé

11. Glossaire

Terme	Définition
Alphabet (Σ)	Ensemble des lettres autorisées.
Langage	Ensemble de mots (ici : les mots acceptés par un automate).
DFA / NFA	Automate fini déterministe / non déterministe.
Epsilon (ε)	Mot vide ; une transition epsilon ne consomme aucune lettre.
État accessible / coaccessible / utile	Atteignable depuis l'initial / permettant d'atteindre un final / les deux.
Émondé (trim)	Automate dont tous les états sont utiles.
Puits	État poubelle d'où l'on ne peut plus accepter. *Implicite* : représenté par une transition manquante.
Complet	Chaque état a une transition pour chaque lettre.
Déterminisation	Conversion NFA → DFA par sous-ensembles.
Minimisation	DFA ayant le moins d'états pour un langage donné ; unique à un renommage près.
Myhill-Nerode	Théorème reliant états distinguables et automate minimal ; classes d'états ou de mots équivalents.
Dérivée de Brzozowski	Quotient à gauche d'une regex par une lettre.
Lemme d'Arden	Solution de X = A·X + B : X = A*·B.
SCC	Composante fortement connexe : états mutuellement atteignables.
Grammaire régulière	Règles de la forme A → aB ou A → ε ; équivalente aux automates finis.
Monoïde syntaxique	Structure algébrique des comportements des mots sur l'automate.
Mixin	Classe qui apporte des méthodes à combiner par héritage multiple.
MRO	Ordre dans lequel Python cherche une méthode en héritage multiple ; détermine ce que *super()* atteint.
Wrapper	Méthode qui délègue à automata-lib puis ré-enveloppe le résultat.
Immuabilité	Un objet n'est jamais modifié ; on en crée un nouveau.
Fixture	Élément réutilisable préparé pour des tests pytest.
Property-based testing	Tester une propriété sur de nombreuses entrées générées aléatoirement.
Differential testing	Comparer deux implémentations indépendantes du même algorithme.
Couverture	Part du code exécutée par les tests.
Principe de Liskov	Une sous-classe doit accepter tout ce que sa classe parente accepte.
PEP8	Guide de style officiel de Python.
venv	Environnement virtuel Python isolé.